Suppressing the original source data in the response

Sometimes we can avoid clogging the network and wasting bandwidth by not retrieving source data that is not required. For example, imagine if our source document has 500 fields but not all of them are relevant to retrieve. We can control what fields are sent to the client and which are not by tweaking the fields in a response.

We can also set the flag `"_source": false` on the request to completely suppress sending the source data in the response. The flag is set at the root level (the same as at the query level):

```
GET books/_search
{
  "_source": false,          <-- Suppressing the source
  "query": {
    "ids": {
      "values": [1,2,3]
    }
  }
}
```

Of course, this way of retrieving the documents is cumbersome. Imagine if we wanted to retrieve 1,000 documents with 1,000 IDs! Well, fortunately, we don't need to use an `ids` query. Instead, we can utilize other search features that we go over later in the book.

So far, we've looked at a document API that fetches documents using IDs. This is not exactly a search feature but rather data retrieval. Elasticsearch provides a myriad of search features to fetch results based on various conditions and criteria. We have dedicated several chapters to search, from basic to advanced, later in the book. But as curious developers, we want to see high-level search queries in action, don't we? Let's discuss the set of queries based on search functions.

RETRIEVING ALL DOCUMENTS

In the previous section, we worked with the basic `_search` endpoint. Using the same syntax, we can write a query to fetch *all* the books from our index.

Listing 2.6 Retrieving all documents from the `books` index

```
GET books/_search          <-- The URL doesn't need a body.
```

As you can see, when you search for all records, you don't need to attach the body to the query (see the sidebar on the `match_all` query, which explains this query's syntax). The response, shown in figure 2.12, returns the three documents in our `books` index as expected.

Don't worry if you are baffled with the query syntax or the response fields. We cover the semantics in the coming chapters.